

JavaScript Web Workers

[Back to JS page](#)

Table of Contents

- [JavaScript Web Workers](#)
 - [Creating a Worker](#)
 - [Example 1](#)
 - [Sending and Receiving Messages](#)
 - [Example 2](#)
 - [Starting and Stopping a Worker](#)
 - [Example 3](#)
 - [Worker Errors](#)
 - [Example 4](#)
 - [Using Timers Inside a Worker](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

Why Web Workers?

JavaScript is single-threaded. Long-running tasks block the UI. Web Workers run JavaScript in the background, on a separate thread, without interfering with the user interface.

A Web Worker is a JavaScript file that runs in the background, independently of other scripts.

Creating a Worker

To create a Web Worker, write a separate JavaScript file and create a `Worker` object:

worker.js - The worker file that runs in the background:

```
onmessage = function(e) {  
  let result = e.data * 2;  
  postMessage("Result: " + result);  
};
```

Main page:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Creating a Worker</h4>
<button onclick="startWorker()">Start Worker</button>
<p id="demo"></p>

<script>
let worker;

function startWorker() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(worker) == "undefined") {
      worker = new Worker("ex1-create_worker/worker.js");
    }
    worker.onmessage = function(event) {
      document.getElementById("demo").innerHTML = event.data;
    };
  } else {
    document.getElementById("demo").innerHTML = "Sorry, Web Workers are not supported.";
  }
}
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Sending and Receiving Messages

Communication between the main page and a worker uses `postMessage()` and `onmessage` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Sending and Receiving Messages</h4>
<button onclick="sendMessage()">Send Message to Worker</button>
<p id="demo"></p>

<script>
let worker2;

function sendMessage() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(worker2) == "undefined") {
      worker2 = new Worker("ex2-messages/worker.js");
    }
    worker2.onmessage = function(event) {
      document.getElementById("demo").innerHTML = event.data;
    };
    worker2.postMessage("Hello Worker!");
  } else {
    document.getElementById("demo").innerHTML = "Web Workers not supported.";
  }
}
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Starting and Stopping a Worker

You can stop a worker using `worker.terminate()` :

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Start and Stop a Worker</h4>
<button onclick="startWorker3()">Start Worker</button>
<button onclick="stopWorker3()">Stop Worker</button>
<p id="demo"></p>

<script>
let worker3;

function startWorker3() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(worker3) == "undefined") {
      worker3 = new Worker("ex3-start_stop/worker.js");
    }
    worker3.onmessage = function(event) {
      document.getElementById("demo").innerHTML = event.data;
    };
  } else {
    document.getElementById("demo").innerHTML = "Web Workers not supported.";
  }
}

function stopWorker3() {
  if (worker3) {
    worker3.terminate();
    worker3 = undefined;
    document.getElementById("demo").innerHTML = "Worker stopped";
  }
}
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Worker Errors

You can catch errors in workers using `onerror` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Worker Errors</h4>
<button onclick="startWorker4()">Start Worker (with error)</button>
<p id="demo"></p>

<script>
let worker4;

function startWorker4() {
  if (typeof(Worker) !== "undefined") {
    worker4 = new Worker("ex4-errors/worker.js");
    worker4.onmessage = function(event) {
      document.getElementById("demo").innerHTML = event.data;
    };
    worker4.onerror = function(error) {
      document.getElementById("demo").innerHTML =
        "Error in worker: " + error.message + "<br>" +
        "Line: " + error.lineno + "<br>" +
        "File: " + error.filename;
    };
  } else {
    document.getElementById("demo").innerHTML = "Web Workers not supported.";
  }
}
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Using Timers Inside a Worker

Workers can use `setTimeout()` and `setInterval()` :

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Timers Inside a Worker</h4>
<button onclick="startWorker5()">Start Timer Worker</button>
<button onclick="stopWorker5()">Stop Timer Worker</button>
<p id="demo"></p>

<script>
let worker5;

function startWorker5() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(worker5) == "undefined") {
      worker5 = new Worker("ex5-timers/worker.js");
    }
    worker5.onmessage = function(event) {
      document.getElementById("demo").innerHTML = event.data;
    };
  } else {
    document.getElementById("demo").innerHTML = "Web Workers not supported.";
  }
}

function stopWorker5() {
  if (worker5) {
    worker5.terminate();
    worker5 = undefined;
    document.getElementById("demo").innerHTML = "Timer worker stopped";
  }
}
</script>

</body>
</html>

```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Web Workers](#)